COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S2
DATE PERFORMED:	17/07/2026	**DATE SUBMITTED:**	07/17/2026
NAME:	Julian Benedict D.R. Barizo	**INSTRUCTOR:**	Jimlord Quejado

ACTIVITY NO. 2.1 **[Arrays, Pointers and Dynamic Memory Allocation]**

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge=18) {
            studentName = std::move (newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        }

        //deconstructor
        ~Student() {
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent) {
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Display Attributes
        void printDetails () {
            std::cout << this->studentName << " " << this->studentAge << std::endl;
        }
};

int main () {
    Student student1("Roman", 28);
    Student student2(student1);
    Student student3;
    student3 = student2;

    return 0;
}
```

```
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observation

I've observed that it initializes three student variables from the Student class. It initializes student 1 as Roman with the age of 28 using the constructor, hence why the output says that the constructor was called once. Secondly, the output says that the copy constructor was called because student2's data copied student1's, and student3 copied student2 which copied student1. Lastly, three destructors were called to free up memory from the three student

	variables that were initialized.

Table 2.1. Initial Driver Program

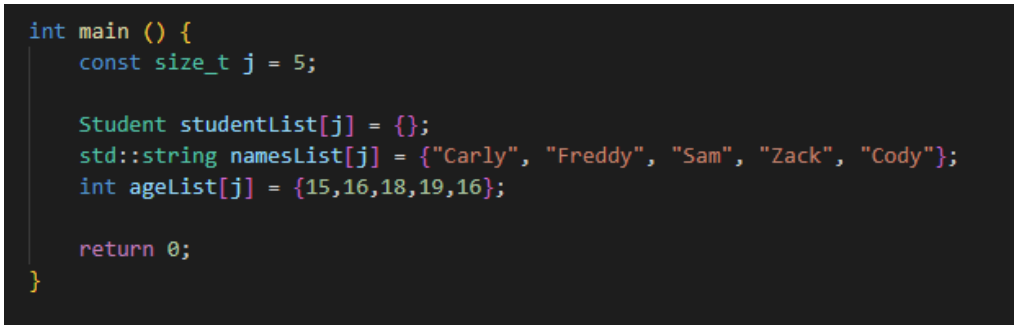
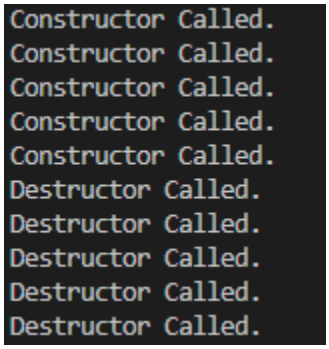
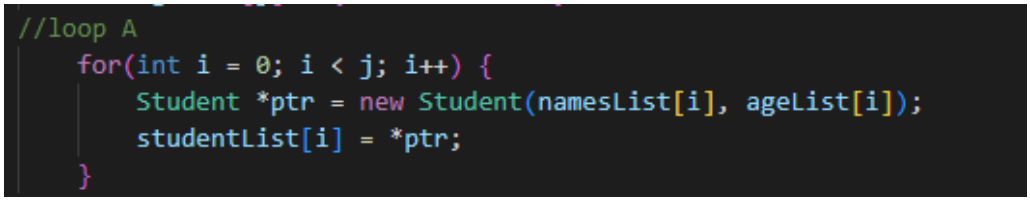
Screenshot	 
Observation	The output says that the constructor was called upon five times because of the size of the student list array to make room for the names and ages of the five students for their data to be put in the array. The destructor was also called five times to delete the student data like their name and age to allocate space and free up memory.

Table 2.2. Modified Driver Program with Student Lists

Loop A	
Observation	For this loop, what I've observed was that this part of the code adds the student's names and list into the array by incrementing until the array is filled with their respective data and only then will the for loop come to a stop.

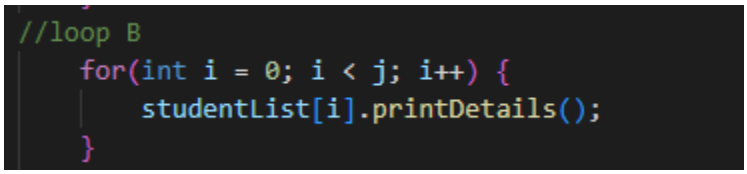
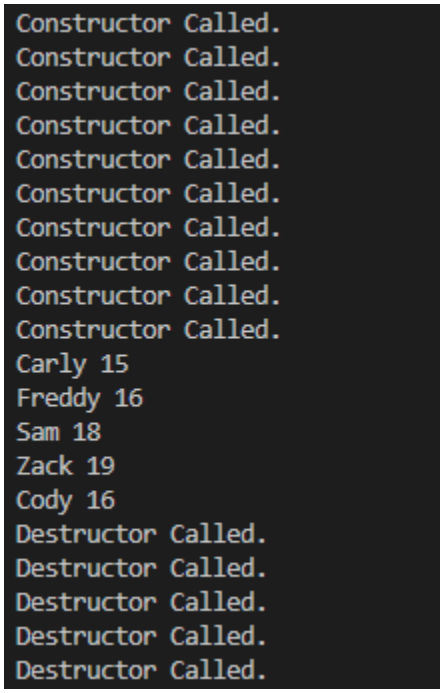
Loop B	
Observation	For this loop, what I've observed was that after it is done executing Loop A which puts their data in the array. This part prints their data contained inside the array and keeps looping until it outputs all their information..
Output	
Observation	From what I've observed on the output, the constructor got called 10 times because it initialized the placeholder data "John Doe" and "18" first before the actual names of the students. After that, because of Loop B it managed to output their name and age before the destructor getting called to allocate some space.

Table 2.2. Modified Driver Program with Student Lists

Modifications

```cpp
#include <iostream>
#include <string.h>

class Student{
    private:
        std::string studentName;
        int studentAge;
    public:
        //constructor
        Student(std::string newName = "John Doe", int newAge=18) {
            studentName = std::move (newName);
            studentAge = newAge;
            std::cout << "Constructor Called." << std::endl;
        }

        //deconstructor
        ~Student() {
            std::cout << "Destructor Called." << std::endl;
        }

        //Copy Constructor
        Student(const Student &copyStudent) {
            std::cout << "Copy Constructor Called" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
        }

        //Copy Assignment
        Student& operator=(const Student& copyStudent){
            std::cout << "Copy Assignment Constructor Called.\n" << std::endl;
            studentName = copyStudent.studentName;
            studentAge = copyStudent.studentAge;
            return *this;
        }

        //Display Attributes
        void printDetails () {
            std::cout << this->studentName << " " << this->studentAge << std::endl;
        }

};
```

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Copy Assignment Constructor Called.

Constructor Called.
Copy Assignment Constructor Called.

Constructor Called.
Copy Assignment Constructor Called.

Constructor Called.
Copy Assignment Constructor Called.

Constructor Called.
Copy Assignment Constructor Called.

Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

Observations	In here, I added the missing Copy assignment operator because when I was using the code visualizer, I noticed that it wasn't copying the full data of the students because the assignment operator was missing. After executing, it managed to call the copy constructor and successfully now copies the data of the students.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include <string>

class Item {
protected:
    std::string name;
```

```
    double price;
    int quantity;

public:
    // Default / Parameterized Constructor
    Item(std::string n = "", double p = 0.0, int q = 0)
        : name(n), price(p), quantity(q) {}

    // Virtual Destructor
    virtual ~Item() {}

    // Copy Constructor
    Item(const Item& other) {
        name = other.name;
        price = other.price;
        quantity = other.quantity;
    }

    // Copy Assignment Operator
    Item& operator=(const Item& other) {
        if (this != &other) {
            name = other.name;
            price = other.price;
            quantity = other.quantity;
        }
        return *this;
    }

    // Function to calculate total item cost
    double calculateSum() const {
        return price * quantity;
    }

    // Display details
    virtual void display() const = 0;

    // Getters
    std::string getName() const { return name; }
```

```
};

// Fruit Class
class Fruit : public Item {
public:
    Fruit(std::string n = "", double p = 0.0, int q = 0) : Item(n, p, q)
{}

    ~Fruit() override {}

    Fruit(const Fruit& other) : Item(other) {}

    Fruit& operator=(const Fruit& other) {
        if (this != &other) {
            Item::operator=(other);
        }
        return *this;
    }

    void display() const override {
        std::cout << "[Fruit]     " << name << " | Price: PHP " << price
                  << " | Qty: x" << quantity
                  << " | Total: PHP " << calculateSum() << "\n";
    }
};

// Vegetable Class
class Vegetable : public Item {
public:
    Vegetable(std::string n = "", double p = 0.0, int q = 0) : Item(n,
p, q) {}

    ~Vegetable() override {}

    Vegetable(const Vegetable& other) : Item(other) {}

    Vegetable& operator=(const Vegetable& other) {
        if (this != &other) {
```

```
            Item::operator=(other);
        }
        return *this;
    }

    void display() const override {
        std::cout << "[Vegetable] " << name << " | Price: PHP " << price
                  << " | Qty: x" << quantity
                  << " | Total: PHP " << calculateSum() << "\n";
    }
};

// Problem 3: Function to calculate total sum of all objects in the list
double TotalSum(Item* GroceryList[], int size) {
    double total = 0.0;
    for (int i = 0; i < size; ++i) {
        if (GroceryList[i] != nullptr) {
            total += GroceryList[i]->calculateSum();
        }
    }
    return total;
}

// Helper function to display the entire list
void displayGroceryList(Item* GroceryList[], int size) {
    std::cout << "-------------------------------------------------\n";
    for (int i = 0; i < size; ++i) {
        if (GroceryList[i] != nullptr) {
            GroceryList[i]->display();
        }
    }
    std::cout << "-------------------------------------------------\n";
}

int main() {
    int size = 4;

    // Problem 2: Create dynamic instances and store pointers in
```

```
GroceryList array
    Item** GroceryList = new Item*[size];

    GroceryList[0] = new Fruit("Apple", 10.0, 7);
    GroceryList[1] = new Fruit("Banana", 10.0, 8);
    GroceryList[2] = new Vegetable("Broccoli", 60.0, 12);
    GroceryList[3] = new Vegetable("Lettuce", 50.0, 10);

    std::cout << "=== Jenna's Initial Grocery List ===\n";
    displayGroceryList(GroceryList, size);

    // Problem 3: Calculate and display Total Sum
    std::cout << "Total Sum: PHP " << TotalSum(GroceryList, size) <<
"\n\n";

    // Problem 4: Delete Lettuce and de-allocate memory
    std::cout << "=== Removing 'Lettuce' from Grocery List... ===\n";
    for (int i = 0; i < size; ++i) {
        if (GroceryList[i] != nullptr && GroceryList[i]->getName() ==
"Lettuce") {
            delete GroceryList[i];      // Free allocated object
            GroceryList[i] = nullptr;   // Set pointer to nullptr
        }
    }

    std::cout << "Updated List:\n";
    displayGroceryList(GroceryList, size);

    // Recalculate total
    std::cout << "Updated Total Sum: PHP " << TotalSum(GroceryList,
size) << "\n\n";

    // Clean up remaining allocated memory
    for (int i = 0; i < size; ++i) {
        delete GroceryList[i];
    }
    delete[] GroceryList;
```

```
    return 0;
}
```

Conclusion
Through this activity, I learned the importance of pointers, arrays, and dynamic memory allocation in creating flexible and efficient programs, especially when managing objects through polymorphism. I also gained a better understanding of how properly initializing classes, constructors, destructors, copy constructors, and copy assignment operators allows the program to allocate, copy, and release memory correctly while producing the expected output. Overall, I believe I performed well in this activity because I was able to implement the required concepts and understand how dynamic memory and object-oriented programming work together, although I still found some parts challenging. My main areas for improvement are developing a deeper understanding of how classes interact with one another and becoming more confident in writing and initializing constructors, destructors, copy constructors, and copy assignment operators without relying heavily on references.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Asked the AI to explain how the code will work and what its final output is going to be. Also assisted in some syntax errors and overall initialization of code**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.